

SIH PS 26117 — Technical Developer Guide

1. Problem Definition

We are building a **self-hosted, on-premise Agentic AI Workbench** for confidential industrial environments such as refineries, PSUs, defence-linked organizations, and government offices.

The system must provide capabilities similar to modern AI assistants while ensuring:

- **No confidential data leaves the organization's infrastructure**
- Multiple **open-weight AI models** can run locally
- The correct model is selected automatically for each task
- AI can perform **multi-step agentic workflows**
- AI can use controlled local tools
- Documents, scans, images, P&IDs and other multimodal inputs are supported
- Internal SOPs, manuals and documents can be searched through local RAG
- AI can create real deliverables such as DOCX, PPTX, XLSX and code
- Generated code can be executed inside a secure sandbox
- All important actions are auditable
- Network isolation can be visibly demonstrated

The prototype should work on a **single workstation/server with a mid-range GPU**, even when the internet is unavailable.

2. High-Level Architecture



The system is essentially:

UI → Backend → Agent → Models + Tools + Knowledge + Sandbox → Verified Deliverable

3. Local Model Runtime

Do not build the application around one specific model.

Use a **Local Model Gateway** so models can be added or replaced without changing the rest of the application.

Possible runtimes:

- Ollama
- llama.cpp
- vLLM
- Hugging Face Transformers
- TensorRT-LLM

Possible model roles:

Model	Responsibility
General/Reasoning LLM	Analysis, planning, document reasoning
Coding LLM	Code generation and debugging
Vision/Multimodal LLM	Images, scans, P&IDs

The application communicates with the **gateway**, not directly with individual models.

4. Model Registry

Maintain a registry containing:

 TEXT
Model ID Model name Model type Context length Quantization VRAM requirement Supported modalities Supported tasks Availability Priority Endpoint

Example:

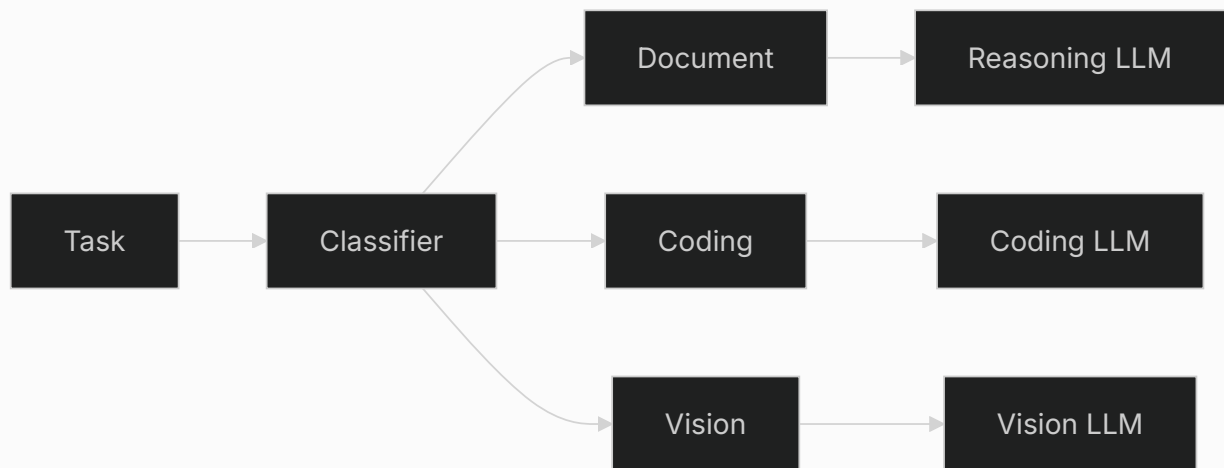
JSON

```
{
  "id": "coding-llm",
  "type": "coding",
  "capabilities": ["python", "debugging", "testing"],
  "endpoint": "local://coding-model"
}
```

This makes the system **model-agnostic**.

5. Model Routing

The router determines which model should handle a request.



Start with a simple hybrid router:

1. Detect task type
2. Check model capabilities
3. Check model availability
4. Select the best available model

Do not over-engineer model routing for the prototype.

6. Agent Architecture

The central component is the **Agent Orchestrator**.

It should operate as:

TEXT

User Request

↓

Understand

↓

Plan

↓

Select Model / Tools

↓

Execute

↓

Observe Result

↓

Verify

↓

Retry if necessary

↓

Final Answer / Artifact

The agent should maintain a task state containing:

TEXT

task_id

user_request

plan

current_step

tool_calls

observations

artifacts

errors

verification_status

final_result

7. Tool System

Tools must have controlled interfaces and permissions.

File tools

TEXT

read_file

write_file

list_files

search_files

Document tools

 TEXT

```
extract_pdf_text
ocr_document
create_docx
create_pptx
create_xlsx
```

Knowledge tools

 TEXT

```
search_knowledge_base
retrieve_document
retrieve_section
```

Code tools

 TEXT

```
create_code
execute_code
run_tests
```

Vision tools

 TEXT

```
analyze_image
analyze_pdf_page
```

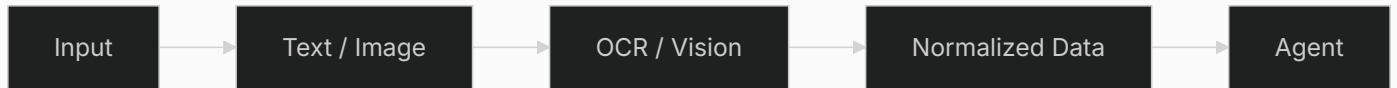
Never give the agent unrestricted OS access.

8. Multimodal Processing

Supported inputs:

- PDF
- Scanned PDF
- Image
- Photograph
- Engineering drawing
- P&ID
- Handwritten document

Processing should roughly follow:



For scanned PDFs:

📄 TEXT

```
PDF
↓
Detect text layer
↓
If scanned → Render pages
↓
OCR / Vision
↓
Normalize extracted content
↓
Agent
```

All OCR and vision processing must remain local.

Possible OCR technologies:

- Tesseract
- PaddleOCR
- EasyOCR
- Local VLM

9. Local RAG

The organization should be able to load:

📄 TEXT

```
SOPs
Engineering manuals
Safety procedures
Past reports
Internal policies
Technical documentation
Past correspondence
```

Ingestion:

📄 TEXT

Documents

↓

Parser

↓

Chunking

↓

Local Embeddings

↓

Vector Database

Retrieval:

📄 TEXT

User Request

↓

Agent

↓

Knowledge Search

↓

Relevant Chunks

↓

Reasoning Model

The RAG system should return **source information**.

Example:

📄 TEXT

Recommendation:

Inspection should be rejected because the observed condition violates the specified tolerance.

Source:

SOP-ENG-042

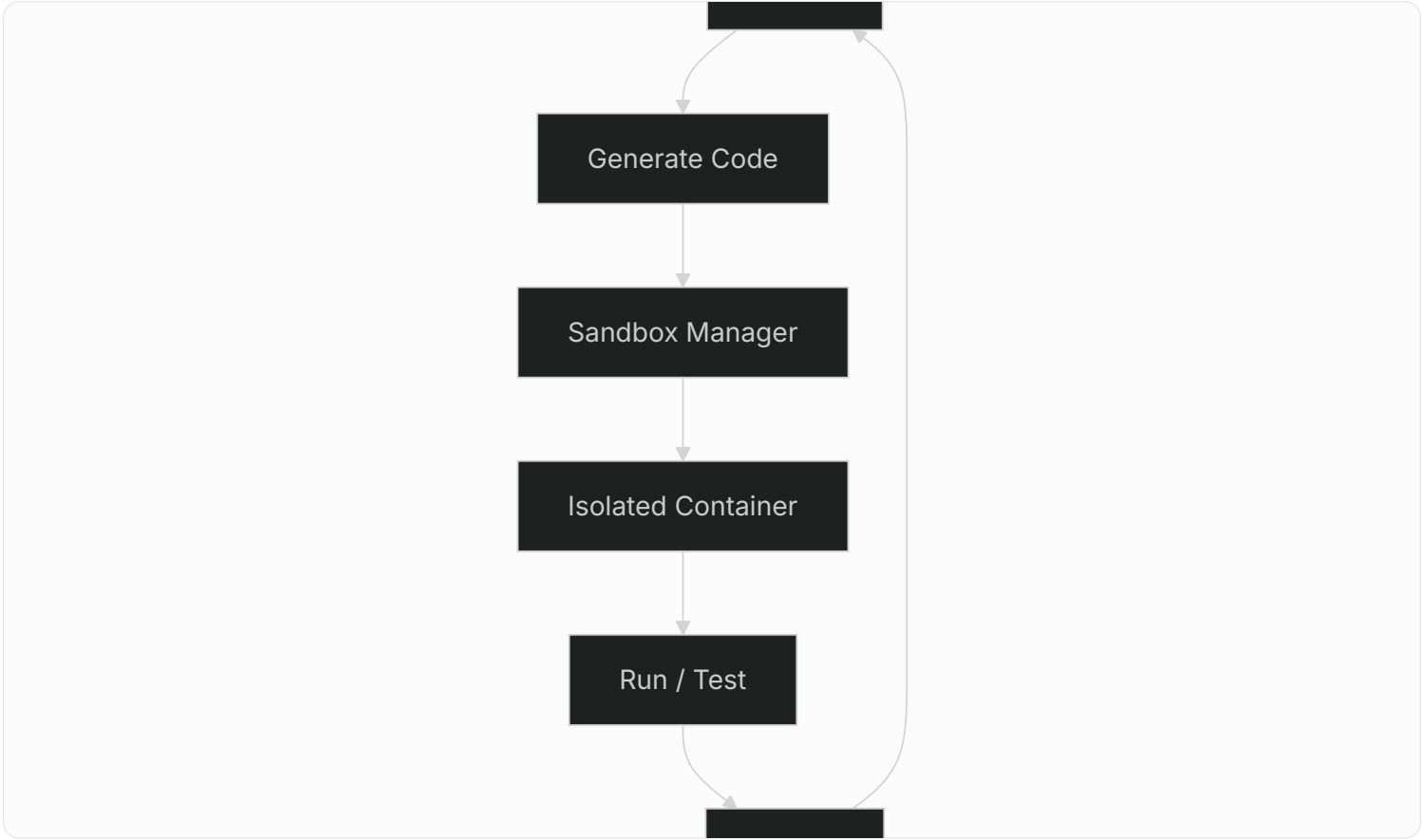
Section 4.2

Page 17

This provides traceability and reduces unsupported AI claims.

10. Sandboxed Code Execution

Generated code must never execute directly on the host.



Sandbox requirements:

- No network
- CPU limit
- RAM limit
- Execution timeout
- Restricted filesystem
- Temporary workspace
- Process isolation

Example workflow:

📄 TEXT

```
Generate code
↓
Execute
↓
Tests fail?
├ Yes → Fix → Execute again
└ No  → Return verified result
```

11. Artifact Generation

The system must produce actual deliverables, not only chat responses.

Initial targets:

TEXT

.docx

.pptx

.xlsx

.pdf

.py

.csv

.txt

Example:

TEXT

Inspection Report

↓

Agent Analysis

↓

SOP Retrieval

↓

Recommendation

↓

DOCX Generator

↓

approval_note.docx

↓

Validation

The frontend should expose generated artifacts separately from the conversation.

12. Verification

Every important output should be validated where possible.

Documents

TEXT

Generate

↓

Check file validity

↓

Check required sections

↓

Return

Excel

TEXT

Generate

↓

Validate workbook

↓

Check formulas

↓

Check required fields

Code

TEXT

Generate

↓

Execute

↓

Test

↓

Fix if required

↓

Verified result

The agent should be able to iterate when verification fails.

13. Human-in-the-Loop

The AI should **prepare and recommend**, not silently make sensitive organizational decisions.

Analyze



Recommend



Generate Artifact



Human Review



Approve / Modify / Reject

For example:

TEXT

Inspection Report

↓

AI Analysis

↓

Approval Recommendation

↓

Approval Note

↓

Human Review

14. Sovereignty / Air-Gapped Operation

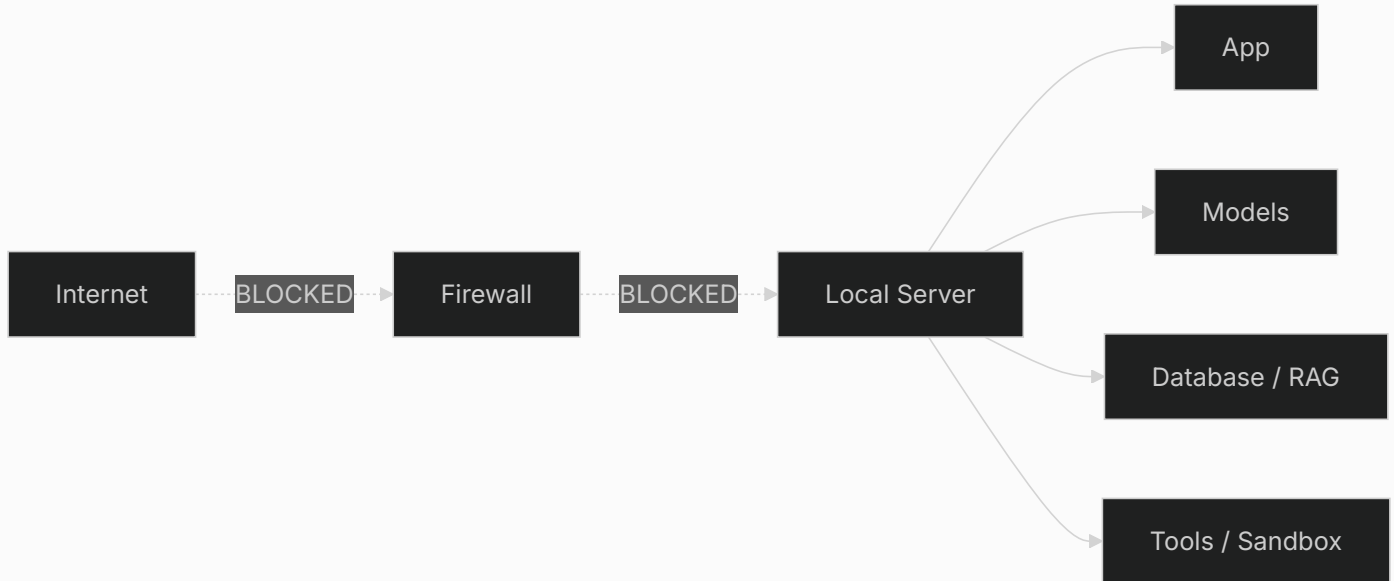
This is a core requirement, not an optional security feature.

The system should continue working when:

TEXT

Internet = DISABLED

Architecture:



There must be no runtime dependency on cloud APIs, cloud OCR, cloud embeddings, cloud telemetry or external model endpoints.

15. Sovereignty Verification

Do not merely display:

"100% Local"

The infrastructure must enforce and demonstrate it.

Use:

TEXT

tcpdump
Wireshark
iptables / nftables
ss

The UI can additionally show:

TEXT

SOVEREIGNTY MONITOR

Internet Access	BLOCKED
External API Calls	0
External DNS Queries	0
External Connections	0
Local Requests	184

Status: AIR-GAPPED

The network layer should actually prevent external communication.

16. Security

Required controls:

- Local authentication
- Role-based authorization
- File permissions
- Tool permissions
- Sandbox isolation
- Network isolation
- Audit logging
- No external telemetry
- Local-only inference

Basic security flow:

TEXT

```
User
↓
Authentication
↓
Authorization
↓
Agent
↓
Controlled Tools / Models
↓
Audit Log
```

17. Audit Logging

Log important agent operations:

JSON

```
{
  "timestamp": "...",
  "task_id": "...",
  "agent_action": "search_knowledge_base",
  "tool": "kb_search",
  "input": "...",
  "result": "...",
  "model": "local-reasoning-model"
}
```

The UI should expose an execution timeline:

TEXT

- ✓ Read inspection_report.pdf
- ✓ OCR completed
- ✓ Retrieved SOP-042
- ✓ Analyzed findings
- ✓ Generated approval note
- ✓ Validated DOCX

This is useful for:

- Debugging
- Transparency
- Security auditing
- Demonstrating agentic behavior to judges

18. Backend Structure

A practical structure:

```

backend/
├── api/
│   ├── chat
│   ├── files
│   ├── tasks
│   ├── artifacts
│   └── monitoring
├── agent/
│   ├── planner
│   ├── executor
│   ├── memory
│   └── verifier
├── models/
│   ├── registry
│   ├── router
│   └── adapters
├── tools/
│   ├── filesystem
│   ├── documents
│   ├── knowledge
│   ├── code
│   └── vision
├── rag/
│   ├── ingestion
│   ├── embeddings
│   ├── retrieval
│   └── citations
├── sandbox/
│   ├── executor
│   └── limits
├── artifacts/
│   ├── docx
│   ├── pptx
│   ├── xlsx
│   └── pdf
└── security/
    ├── auth
    ├── permissions
    ├── audit
    └── network

```

19. Frontend

The frontend should behave like an **AI workbench**, not a conventional admin dashboard.

Main areas:

TEXT

Workspace

- New Task
- Documents
- Knowledge
- Artifacts
- Models
- Activity

Main Area

- Conversation

Context

- Files
- Sources
- Artifacts

Bottom

- Agent Execution Timeline

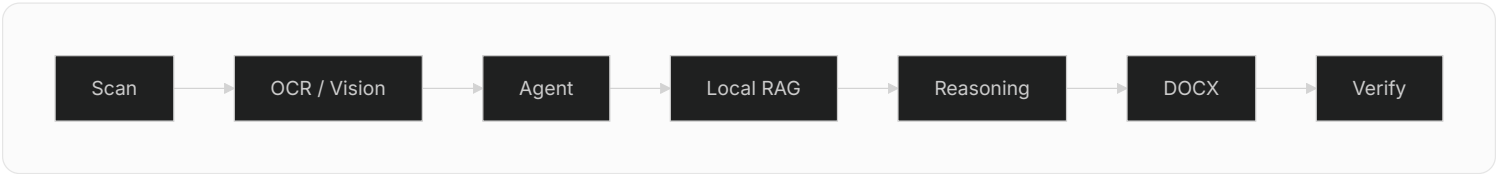
Important UI concepts:

- Conversation
- Task/workspace
- File attachments
- Agent execution timeline
- Tool calls
- Sources
- Generated artifacts
- Model information
- Sovereignty status

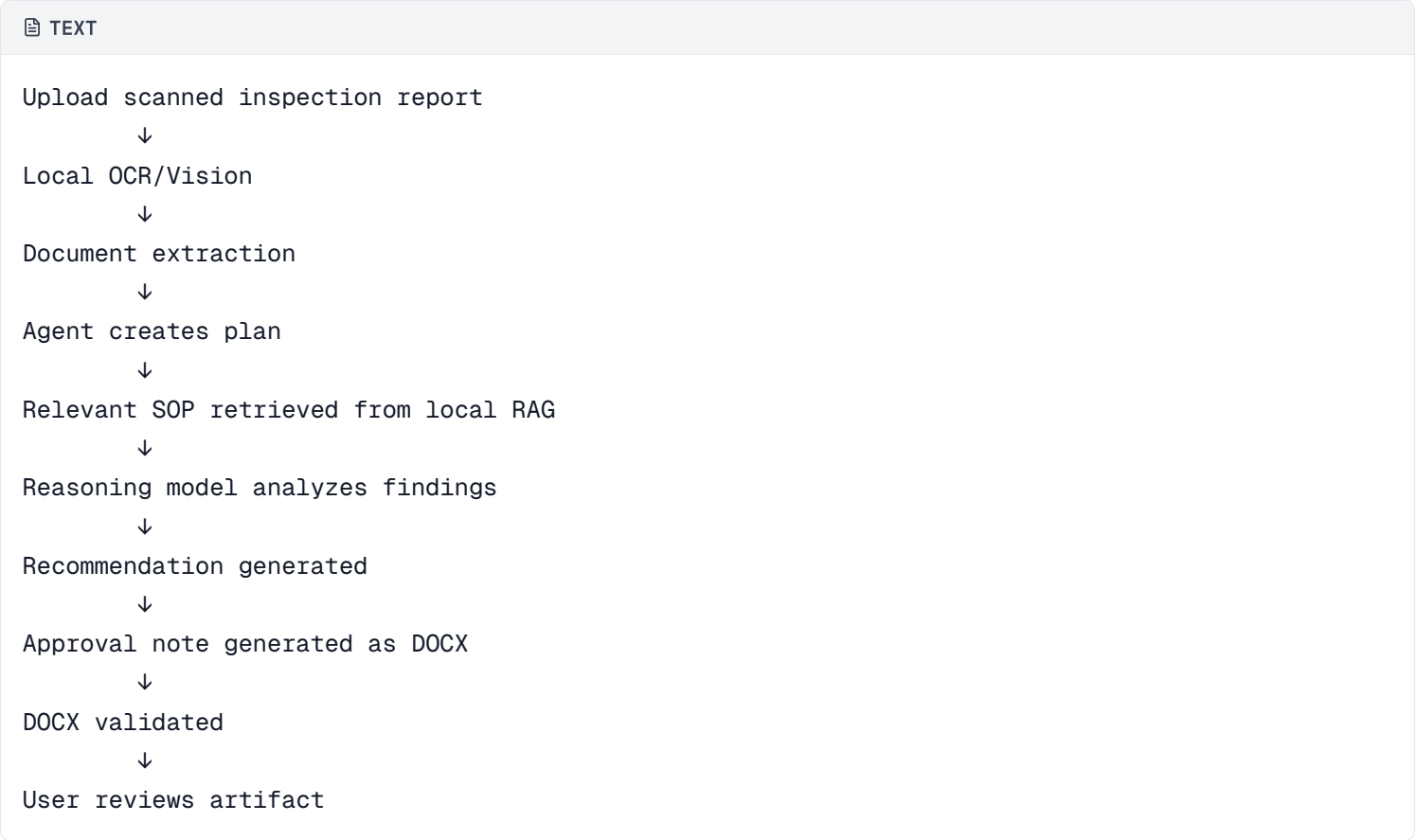
20. Primary Demo

The primary SIH demo should be:

Scanned Inspection Report → Approval Note



Detailed workflow:



This single workflow demonstrates:

- Multimodal AI
- Agentic execution
- Model routing
- RAG
- Tool calling
- Document generation
- Verification
- Local processing

21. Secondary Demo

Coding Agent

User:
"Create a Python program to calculate X."

↓

Task Router

↓

Coding Model

↓

Generate Code

↓

Sandbox

↓

Run Tests

↓

FAIL → Fix → Run Again

↓

PASS

↓

Final Result

This demonstrates genuine agentic iteration.

22. Third Demo

P&ID / Industrial Image Understanding



Example:

Identify the major components visible in this P&ID and explain the flow path.

Use publicly available industrial samples, as specified by the problem statement.

24. Development Order



Recommended order:

1. Local inference
2. Backend API
3. Model abstraction + registry
4. Model router
5. Agent planner/executor
6. Tool system
7. Local RAG
8. OCR/Vision
9. Sandbox
10. Artifact generation
11. Security + network isolation
12. Frontend polish

Do not attempt to build every feature simultaneously.

25. Core Engineering Principle

"We need to build a secure execution environment around local AI models."

The core technology stack is:

TEXT

```
Local Models
+
Model Routing
+
Agentic Tools
+
Multimodal Processing
+
Local RAG
+
Artifact Generation
+
Sandboxed Execution
+
Verification
+
Auditability
+
Sovereignty
```

Responsibility of each layer

Layer	Responsibility
Models	Intelligence and reasoning
Router	Select appropriate model
Agent	Plan and execute tasks
Tools	Give the agent capabilities
RAG	Provide organizational knowledge
Vision/OCR	Understand non-text inputs
Sandbox	Safely execute generated code
Artifacts	Produce usable deliverables
Verification	Validate results
Security	Control access and execution
Sovereignty Layer	Prevent external communication
Audit	Make operations traceable

The final product is therefore an on-premise AI workbench where confidential industrial data can be understood, reasoned over, acted upon, verified, and converted into real deliverables without leaving the organization's infrastructure.